ASSIGNMENT 2 – CO2

Object-Oriented Programming with C++

Functions and Object-Oriented Concepts

Item	Details
Student name	Syed Nazma
Register number	192425193
Department	AI&ML
Compiler and version	g++ (Ubuntu 13.3.0), C++17 on Ubuntu 24.04
Compile command	g++ -std=c++17 -Wall -Wextra -o name name.cpp
GitHub repository	https://github.com/nazma824/DSA0117

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

Hostel Fee Concession Updater

The hostel office gives merit (10%), sports (15%) and single-parent (25%) concessions. A function that works on a copy prints the right fee inside the function, but the stored fee of the student is unchanged. The task is to write the function three ways and to explain and defend the difference.

1.1 The three implementations

The source file is Q1/three_ways.cpp (in the repository). A fourth function returns the new fee, and a large Student structure is used to compare the cost of passing objects.

Prediction made before running (initial fee 45000, 10% concession each time)

Version	**Predicted value inside**	**Predicted stored fee after the call**	**Reason**
By value	40500	45000	Only the copy is reduced
By reference	40500	40500	The reference is the caller's variable
By pointer (on 40500)	36450	36450	The pointer holds the caller's address

All predictions matched the observed results. The by-value function printed the correct fee inside the function, which is exactly the symptom reported by the hostel office, but the stored fee stayed at 45000.

1.2 Inquiry – where each version stores its parameter

- **By value:** the callee has its own new variable (a copy of the 8-byte long long) in its stack frame. Changing it cannot affect the caller's variable, which is a different object.
- **By reference:** no new object is created in the language; the parameter is another name (alias) for the caller's variable. A compiler usually implements it with a hidden address, but every use of the name is a use of the caller's object.

- **By pointer:** the callee has its own local pointer variable (8 bytes on this system) holding the address of the caller's variable. The pointer itself is a copy, but what it points to is the caller's object, so *fee changes the caller's fee. A pointer can be null, so it must be checked.

Only the reference and pointer versions change the caller's fee because they reach the caller's object. The value version works on a separate copy that is destroyed when the function returns. A **const reference** should be used when the function only reads the argument and the argument is large or expensive to copy (a Student object or a std::string), because it avoids the copy and the compiler rejects accidental changes. For small built-in types such as int or long long, passing by value is as cheap as passing an address and is simpler.

1.3 Design decision – change in place or return the new fee

Decision: the concession function should **return the new fee** (long long applyConcession(long long fee, int percent)) and the caller stores the result. A thin wrapper that takes a Student by reference can then update the stored record in one place.

- Several concessions can be chained clearly: fee = applyConcession(fee, 10); fee = applyConcession(fee, 15); The run above shows 45000 → 40500 → 34425 → 25819.
- The caller keeps the original fee, which is needed for the receipt and for audit (original fee, each concession, final fee).
- The function has no side effects, so it is easy to test with fixed values and cannot corrupt a record when a mistake is made half-way through a list of concessions.
- The only cost is one 8-byte copy, which is negligible. The order of concessions matters because integer division truncates (34425 × 25 / 100 = 8606.25 becomes 8606), so the office must decide and document the order.

Changing in place through a reference would be preferable only if the function also had to update several fields of the same record together (fee, concession total and a flag), which is why the wrapper on Student is suggested.

There was no difference between the prediction and the actual output. The reasoning is as follows.

- **bigger(a, c) = 100:** a is 1 and c is 13. The condition p > q is false, so the function returns q, a reference to c. The assignment therefore changes c to 100 and a remains 1. Output 1 100.

- **show(1):** y and z both use their defaults (5 and 10), so 1 + 5 + 10 = 16. **show(1, 2):** only z uses its default 10, giving 13. **show(1, 2, 3):** no default is used, giving 6. Each call prints a trailing space, so the line ends with "6 " before the newline.

The call by value leaves the fee at 45000 because the parameter fee in applyByValue is a separate local copy of hostelFee. The statement fee -= fee * percent / 100 reduces that copy to 40500, but the copy is destroyed when the function returns and hostelFee in main was never touched.

1.4 Alternative Concept and Complexity Analysis

If the function returns the new fee instead of changing it through a reference, the solution is:

Version	**Time per call**	**Bytes copied for long long**	**Bytes copied for Student (1160 bytes)**
By value (returns new fee)	O(1)	8	1160 (copy of whole object)
By reference	O(1)	8 (address, may vanish after inlining)	8 (address only)
By const reference	O(1)	8 (address)	8 (address only, read-only)
By pointer	O(1)	8 (address)	8 (address only)

The time is O(1) for every version because the work is one multiplication, one division and one subtraction. The space copied per call is the real difference: a long long costs 8 bytes whichever way it is passed, so the by-value and returning design is the simplest. A Student object of 1160 bytes (measured with sizeof) costs 1160 bytes per call by value but only the size of an address (8 bytes) by reference or const reference. For the Student the const reference is therefore correct when the function only reads the fee, and a non-const reference when it must update it.

Evidence summary: three implementations (three_ways.cpp), predictions before execution, observed results, the returning design with a defended interface, the completed program and the Predict the Output analysis.

Q2 – Return by Reference and Its Dangers (4 Marks)

Examination Marks Register

2.1 Working functions

Q2/register.cpp returns a reference to the highest mark (int&), a const reference (const int&) and a copy (int), and also contains the index and pointer alternatives used in section 2.5.

highest(marks, 6) += 2 changed the array element 91 to 93 directly. Because the function returns int&, the call is an lvalue, so it can also be placed on the left of = (highest(marks, 6) = 90 stored 90 in the array). This works because the returned reference is bound to marks[idx], an object that still exists after the function returns.

2.2 Returning a reference to a local variable

Compiler warning (g++ -std=c++17 -Wall -Wextra dangling.cpp)

```
dangling.cpp: In function 'int& badReference()':
dangling.cpp:5:12: warning: reference to local variable 'local' returned [-Wreturn-local-addr]
    5 |     return local;           // warning: reference to local variable returned
      |            ^~~~~
dangling.cpp:4:9: note: declared here
    4 |     int local = 99;
      |         ^~~~~
```

The variable local has automatic storage: it lives in the stack frame of badReference() and is destroyed when the function returns. The returned reference therefore refers to an object whose lifetime has ended (a dangling reference), and reading through it is undefined behaviour. The standard does not say what happens: in this build the program crashed with a segmentation fault; another compiler or optimisation level could print 99 or any other value, or appear to work and fail later. A warning must never be ignored for this reason.

2.3 Reference, const reference and copy

Return type	What the caller receives	Can the caller change the array element?	Cost and risk
int&	An alias of the element	Yes, even as highest(m, n) = 90	No copy; dangling if the element dies; allows accidental modification
const int&	A read-only alias	No (cr += 1 is a compile error)	No copy; still dangling if the element dies
int	A copy of the value	No; the copy changed to 95 but the array stayed 90	A copy of one int; always safe

2.4 Changed condition – the register is a growing std::vector

Output (g++ -std=c++17)

```
capacity before: 4
capacity after : 128
buffer moved   : true
```

Evidence with AddressSanitizer (g++ -std=c++17 -g -fsanitize=address -DUSE_DANGLING)

```
ERROR: AddressSanitizer: heap-use-after-free on address 0x502000000018
READ of size 4 at 0x502000000018 thread T0
   #0 ... in main Q2/vector_invalid.cpp:25
0x502000000018 is located 8 bytes inside of 16-byte region [0x502000000010,0x502000000020)
freed by thread T0 here:
   #0 ... in operator delete(void*, unsigned long)
```

A std::vector keeps its elements in one contiguous block. When push_back needs more room than the capacity, the vector allocates a larger block, moves the elements into it and frees the old block. The capacity grew from 4 to 128 and the address of the first element changed (buffer moved: true). The reference top, obtained before the growth, still refers to the old, freed block, so using it (top += 2) is a heap-use-after-free, which AddressSanitizer reported at the line of

the use. Any reference, pointer or iterator into a vector is invalid after an operation that reallocates. The response is to obtain the reference again after the vector has changed, to reserve the capacity in advance (reserve) when it is known, or to hold an index instead of a reference.

2.5 Alternative Concept and Complexity Analysis

If the function returns an index or a pointer instead of int&, the calling code is:

Design	**Calling code**	**Time to find**	**Extra space**	**Safety if the array becomes a growing vector**
int&	highest(m,n) += 2;	O(n)	O(1)	Dangling after reallocation
int*	*highestPtr(m,n) += 2;	O(n)	O(1)	Dangling after reallocation; may also be null or out of range
int (index)	m[highestIndex(m,n)] += 2;	O(n)	O(1)	Still valid after growth, because an index is a position and not an address

All three designs scan the array once, so the time is O(n) and the extra space is O(1). The reference and pointer hold an address, and an address into a vector becomes invalid when the vector reallocates, whereas an index remains meaningful as long as the element is not erased. For a register that may grow, returning an index (and indexing at the moment of use) is the safest of the three; the reference is the most convenient when the container cannot reallocate during the lifetime of the reference.

Evidence summary: working functions, the compiler warning (-Wreturn-local-addr), the crash, the dangling-reference explanation, the reallocation evidence with AddressSanitizer and the index/pointer comparison.

Q3 – Default Arguments and Function Overloading (6 Marks)

Courier Charge Calculator

3.1 The ambiguity in the original program

Compiler output (g++ -std=c++17 -Wall -Wextra ambiguous.cpp)

```
ambiguous.cpp:8:24: error: call of overloaded 'charge(double)' is ambiguous
ambiguous.cpp:2:8: note: candidate: 'double charge(double)'
ambiguous.cpp:4:8: note: candidate: 'double charge(double, bool, bool)'
ambiguous.cpp:9:24: error: call of overloaded 'charge(double, bool)' is ambiguous
ambiguous.cpp:3:8: note: candidate: 'double charge(double, bool)'
ambiguous.cpp:4:8: note: candidate: 'double charge(double, bool, bool)'
```

The third function, charge(double, bool = false, bool = false), can be called with one argument, with two arguments or with three. A call charge(2.0) therefore matches both charge(double) and the third function by an exact match on every argument (the default arguments fill the rest), and a call charge(2.0, true) matches both charge(double, bool) and the third function. Neither candidate is better than the other, so the compiler cannot choose and reports an ambiguous call. The defect is that the default arguments make the first two overloads redundant: the same call is possible through two different functions.

3.2 The redesigned functions

The design uses one function per kind of input (grams as int, kilograms as double, a Parcel object). Default arguments are used only inside each function for the optional express and insured flags, and no two overloads accept the same call.

Prediction made before running (which overload is selected)

Call	**Predicted overload**	**Reason**	**Predicted charge**
charge(2)	grams (int)	2 is an int: exact match	0.1
charge(2.0f)	kg (double)	float to double is a promotion; float to int is a conversion; promotion wins	100

Call	Predicted overload	Reason	Predicted charge
charge(2.5)	kg (double)	exact match	125
charge(2500, true)	grams (int)	int exact, bool exact	165
charge(1.5, true, true)	kg (double)	exact on all three	135
charge(parcel)	Parcel	exact match on the class type	190

Observed output

```
charge(2)              -> [grams overload] 0.1
charge(2.0f)           -> [kg overload]    100
charge(2.5)            -> [kg overload]    125
charge(2500, true)     -> [grams overload] 165
charge(1.5, true, true)-> [kg overload]    135
charge(parcel)         -> [Parcel overload]190
```

All six selections and charges matched the predictions. The result for charge(2) is worth noting: the literal 2 is an int, so it means 2 grams and the charge is 0.1, not 100. The call is not ambiguous, but it may surprise a user who meant 2 kg; the interface should document that integers are grams, or the counter should write 2.0 for kilograms. A call such as charge(2L) (a long) is still ambiguous, because long to int and long to double are both conversions of equal rank. This was confirmed with Q3/courier_long.cpp:

```
courier_long.cpp:4:33: error: call of overloaded 'charge(long int)' is ambiguous
courier_long.cpp:2:8: note: candidate: 'double charge(int, bool)'
courier_long.cpp:3:8: note: candidate: 'double charge(double, bool)'
```

3.3 Inquiry – how the compiler chooses, and why defaults go on the right

To resolve a call the compiler first builds the set of functions with that name, then keeps the **viable** ones (right number of arguments, counting defaults, and every argument convertible). For each argument it ranks the conversion needed:

- **Exact match** – same type (or trivial changes such as adding const or turning an array into a pointer). Best.

- **Promotion** – float to double, char or short or bool to int. Better than a conversion.
- **Standard conversion** – int to double, double to int, long to int, pointer conversions. Worse than a promotion.
- **User-defined conversion** – a converting constructor or conversion operator. Worst.

A function wins if it is at least as good as every other viable function on every argument and strictly better on at least one. If no function wins, the call is ambiguous and the program is rejected. This is why charge(2.0f) selects the double version (promotion beats conversion) and why charge(2L) fails (two conversions of equal rank).

Default arguments must be given from the right because arguments are matched to parameters by position, from the left. When a call omits arguments, only the trailing parameters can be missing, because C++ has no syntax to skip a middle argument (a call such as f(1, , 3) does not exist). If a parameter without a default followed one with a default, a call could never supply the later one without the earlier one, so the declaration is rejected. The default value is inserted by the compiler at the call site, so it is chosen at compile time and costs nothing at the point of the call.

3.4 Alternative Concept and Complexity Analysis

Instead of several overloads, one Parcel structure with optional fields, or a function template, could be used:

Design	**Overload resolution cost**	**Bytes copied per call**	**Adding a new option (for example fragile = false)**
Overloads with defaults	Compile time only; no run-time cost	int 4 + bool 1 + bool 1, normally passed in registers; a Parcel by const reference is 8	Every overload gets a new trailing parameter; callers that pass positional bools may silently change meaning; new ambiguities are possible
Single Parcel with optional fields	None (one function)	16 bytes by value (a double and two bools with padding) or 8 by const reference	Add a field with a default member initialiser; existing callers are unaffected

Design	Overload resolution cost	Bytes copied per call	Adding a new option (for example fragile = false)
Function template	Compile time; instantiation per type	Same as the overloads	One place to change, but error messages are longer and the unit rule (grams or kg) is hidden inside the template

Overload resolution happens entirely during compilation, so it adds no time to a running program in any design; the run-time time per call is O(1). The compile-time work grows only with the number of candidate functions and arguments. The space copied per call is a few bytes in all designs. The designs differ mainly in how they cope with change: a Parcel with default member initialisers absorbs a new option without touching old callers, whereas overloads with default arguments must be edited together and keep the risk of ambiguity. The overload design is still the most convenient for the counter staff because common calls are short and the units are chosen by the type of the argument.

Evidence summary: the explained ambiguity, a redesigned set of functions, predicted and observed selections and the reasoning on overload resolution.

Q4 – Specifying a Class with an Array Inside It (6 Marks)

Savings Account Mini-Statement

4.1 Class specification

SavingsAccount has private data: the account number, the balance, an array of 10 Transaction records and a count. The constructor and getBalance() are defined inside the class; record(), deposit(), withdraw() and miniStatement() are declared in the class and defined outside it with the scope resolution operator (SavingsAccount::). The capacity is a single named constant MAXTX so that no number 10 is repeated in the code.

Observed output (12 transactions made, the last 10 are shown)

```
Mini-statement for SB1001 (last 10 transactions)
  Withdrawal 300
  Deposit    400
  Deposit    500
  Withdrawal 600
  Deposit    700
  Deposit    800
  Withdrawal 900
  Deposit    1000
  Deposit    1100
  Withdrawal 1200
  Balance: 6800
withdrawal refused: insufficient balance
```

Hand check: the account opened with 5000; the deposits are 100, 200, 400, 500, 700, 800, 1000, 1100 (total 4800) and the withdrawals are 300, 600, 900, 1200 (total 3000), so the balance is 5000 + 4800 − 3000 = 6800. The first two transactions (100 and 200) were discarded, so the statement starts with the third (withdrawal of 300), as expected.

4.2 Inquiry – why the array is private, and what the 11th transaction does

The array must be private because its meaning depends on rules that only the class knows: count must equal the number of valid entries, the entries must be in order, and the balance must agree with the transactions. If outside code could write tx[] or count directly, it could record a transaction without changing the balance, overwrite history or set count beyond 10 and make

miniStatement() read outside the array. Keeping the data private and exposing only deposit(), withdraw() and miniStatement() lets the class guarantee these rules, and lets the storage change later (for example to a circular buffer) without changing any caller.

Method for the 11th and later transactions	**Work for each new transaction**	**Notes**
Shift the array left by one and store at the end (used above)	Copy MAXTX − 1 entries = 9 copies, so O(k) with k = MAXTX − 1	Entries are always oldest first, so printing is a simple loop; cost grows with the capacity
Circular index: overwrite the oldest entry and advance a start index	One store and one index update, O(1)	Printing must start at start and wrap around; one extra int of data

For 10 entries the cost of shifting is small (about 9 ns per transaction in the measurements of Q7), but it grows in proportion to the capacity, while the circular index costs the same for any capacity.

4.3 Design decision – the bank now wants the last 50 transactions

Decision: change only the constant (static const int MAXTX = 50). This is the design that changes least, because every loop, bound check and array size in the class uses MAXTX and no literal 10 appears anywhere, and the public interface (deposit, withdraw, miniStatement) is unchanged, so no calling code is touched. The memory per account grows from 10 × 16 = 160 bytes to 50 × 16 = 800 bytes (sizeof(Transaction) is 16 with padding). If the shifting version is kept, the cost per transaction grows about five-fold (9 copies become 49); if that matters, the storage can be changed to the circular index inside the class without any change outside it, which is the reason for keeping the array private. The redesign is carried out and measured

4.4 Alternative Concept and Complexity Analysis

The same behaviour with a std::deque, or with a circular buffer and a start index, would be:

Design	**Time to add a transaction**	**Space**	**Remarks**
Fixed array, shift left	O(k), k = MAXTX − 1 once full; O(1) before	MAXTX × 16 bytes, no heap	Simplest; cost grows with capacity
Circular buffer with start index	O(1) always	MAXTX × 16 bytes + one int, no heap	Needs wrap-around in printing
std::deque with pop_front / push_back	O(1) (amortised, may allocate a block)	At most about MAXTX entries plus block bookkeeping, heap memory	Least code, but heap allocation and extra indirection per account

The shifting design costs O(k) per transaction once the array is full, while the circular buffer and the deque cost O(1); all three keep O(MAXTX) space because only the last MAXTX transactions are stored. The measurements for several capacities are given in Q7.

Evidence summary: a complete class specification, member functions defined inside and outside the class, a justified handling of the 11th transaction and a defended design.

Q5 – Static Members and Arrays of Objects (7 Marks)

College Bus Seat Tickets

5.1 From the principle to a class design

The principle is: data that belongs to the whole class, and not to any one object, should exist only once however many objects are created. Applying it to the bus tickets:

Member	Belongs to	Kind	Reason
issued (number of seats given so far)	The whole bus / class	static int (private)	One counter shared by every ticket; each new ticket needs the previous value
CAPACITY (3 seats)	The whole class	static const int	A fixed property of the bus, not of a ticket
count()	The whole class	static member function	The transport office asks without holding any ticket
seat, name	One ticket	Non-static data	Different for each passenger

- **Where a static data member is defined:** a static data member declared in the class is only a declaration. It must be defined exactly once outside the class, in one source file (int BusTicket::issued = 0;), where it receives its storage and initial value. A static const integral member with an in-class initialiser can be used as a constant without this; in C++17 a member can also be declared inline static to define it inside the class.
- **Why a static member function cannot use this or a non-static member:** a static member function is not called on an object (BusTicket::count()), so it has no this pointer. Non-static members exist inside particular objects, so without an object there is nothing for the name seat or name to refer to. It can use only static members and its parameters.
- **Arrays of objects:** BusTicket trip[4] creates four objects, and the constructor runs once for each element in order of the index. Because the constructor is explicit and takes a name, each element needs an initialiser, as in BusTicket trip[4] = {BusTicket("Asha"), ...}. An array without initialisers needs a default constructor.

- **Tickets created in a loop or a function:** each constructor call increments the same static counter, whether the object is created in a loop, in a function or as a temporary. The counter is not decreased when an object is destroyed unless a destructor does so; in this design the seat issued stays issued (shown in section 5.4).

5.2 Predict the Output – Token

My predicted output (written before running)

```
101 103 104 4
```

Actual output

```
101 103 104 4
```

There was no difference. The static member next is defined by int Token::next = 100; and is initialised once, before main starts, so it starts at 100. The constructor runs four times: three times for the elements of counter[3] and once for extra. Each run does ++next and copies the result into id, so the elements receive 101, 102 and 103 and extra receives 104. issued() returns next − 100 = 104 − 100 = 4, which equals the number of constructor calls. The output is counter[0] = 101, counter[2] = 103, extra = 104 and 4.

5.3 Complete the Code – BusTicket

Expected output and actual output (identical)

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

5.4 Test of the design

Test A – an array larger than the capacity (6 tickets for 3 seats), Q5/bus_test_big.cpp. A default constructor was added so that the class can also be used in arrays without initialisers:

```
P1: Seat 1
P2: Seat 2
P3: Seat 3
```

```
P4: Waiting
P5: Waiting
P6: Waiting
Issued: 3 (capacity 3, waiting list = 3)
```

Test B – tickets created inside a function that is called twice, Q5/bus_function.cpp:

```
Call 1
  Asha: Seat 1
  Bala: Seat 2
  count inside function: 2
count after call 1: 2
Call 2
  Chitra: Seat 3
  Dev: Waiting
  count inside function: 3
count after call 2: 3
```

The tickets created inside bookTwo() are destroyed when the function returns, but the static counter lives for the whole program, so the count is not reset: after the first call it is 2 and the second call continues from 2, issuing seat 3 to Chitra and putting Dev on the waiting list. The count is therefore the number of seats issued, not the number of ticket objects alive. If a cancelled ticket should free a seat, a destructor or a cancel() function would have to decrease issued, and the seat numbering would then need more care to avoid issuing the same seat twice.

5.5 Alternative Concept and Complexity Analysis

If the counter were a global variable, or were kept in a separate TicketOffice object passed to every ticket, the solution would be:

Design	**Time per ticket**	**Space for the counter**	**Encapsulation**	**Two buses at once**
Static data member	O(1)	One int shared by all	Private to the class; only the class changes it	No: all tickets share one counter

Design	Time per ticket	Space for the counter	Encapsulation	Two buses at once
Global variable	O(1)	One int	None; any code can change it	No
TicketOffice object	O(1)	One int per office (plus a reference per ticket if stored)	Good: state is inside the office	Yes: one office per bus

Time is O(1) in all three designs (one comparison and one increment) and the space is one counter in each, so performance does not decide the choice. The static member is the best answer when there is exactly one bus, because it is encapsulated and needs no extra argument. The global variable has the same behaviour with no protection. The TicketOffice design is the only one that allows two buses at once and is also easier to test, because each test can create a fresh office, whereas a static counter keeps its value for the whole program and tests depend on the order in which they run.

Evidence summary: a class design derived from the principle, an array of objects, test evidence for six tickets and repeated function calls, and the reasoning on static members.

Q6 – Friend Functions and Inline Functions (8 Marks)

Train Journey Planner

6.1 Design questions

- **Why a friend function when the function works on two objects equally:** minutesBetween(a, b) and earlier(a, b) treat both times alike. A member function has to make one of the two the object on which it is called (this) and the other an argument, which makes the code asymmetrical. A non-member, non-friend function could not read the private h and m at all, and would need public getters. A friend function is an ordinary function that the class has allowed to see its private members, so it can treat the two objects symmetrically without widening the public interface. (For operators whose left operand is not the class, such as 5 + t or an ostream, a member function is not possible at all, which is the other usual reason for friends.)
- **Friend versus member function:** a member function is called through an object (a.minutesTo(b)), has a this pointer and is inside the class scope. A friend is called as an ordinary function (minutesBetween(a, b)), has no this pointer, and receives every object as an explicit parameter. Both can read the private data of the objects they receive. A friend is not a member of the class, is not inherited, and friendship is not transitive.
- **Inline:** the keyword inline is a request to the compiler to replace the call by the body of the function, which removes the cost of the call for very small functions. It is only a request: the compiler may refuse, and it may also inline functions that are not marked inline when optimising. Today its main language effect is that an inline function may be defined in several translation units (a header) without breaking the one-definition rule. Functions defined inside the class body are implicitly inline.
- **What would be lost if h and m were public:** the class could no longer guarantee a valid time. Any code could set h = 37 or m = −5 and every function that trusts the data would be wrong. The internal representation could not be changed (for example to minutes since midnight) without editing all the code that uses h and m. Private members with controlled access keep both safe.

6.2 Complete the Code

Expected output and actual output (identical)

```
205 false
```

Hand check: 9:45 is 9 × 60 + 45 = 585 minutes and 13:10 is 13 × 60 + 10 = 790 minutes, so the difference is 205 minutes. earlier(arrival, departure) asks whether 13:10 is earlier than 9:45, which is false.

6.3 Correctness check

```
11:50 vs 12:10  -> minutes 20, earlier(11:50,12:10)=true, earlier(12:10,11:50)=false
09:45 vs 09:45  -> minutes 0, earlier=false
00:00 vs 23:59  -> minutes 1439, earlier(00:00,23:59)=true
```

Verified by hand: 11:50 = 710 and 12:10 = 730 minutes, difference 20, and 11:50 is earlier than 12:10 across noon; the same time twice gives difference 0 and earlier is false in both orders (a time is not earlier than itself); 00:00 to 23:59 gives 23 × 60 + 59 = 1439.

6.4 Growth investigation – 10,000,000 calls

Q6/time_bench.cpp measures four versions of the same calculation: a friend function defined inline in the header, the same friend defined in another source file (time_out.cpp), so it cannot be inlined without link-time optimisation, a member function, and a non-friend function that uses public getters. The times are pre-built in an array so that the loop measures the call and not the construction of the objects. Compile commands: g++ -std=c++17 -O0 (or -O2) -o bench time_bench.cpp time_out.cpp.

Results (best of 5, milliseconds for 10,000,000 calls; all checksums equal 4790910936)

```
=== -O0
friend, inline          : 60.2056 ms
friend, not inline (TU)  : 57.9984 ms
member function (inline) : 60.1605 ms
public getters (inline)  : 119.425 ms
=== -O2
friend, inline          : 12.5241 ms
friend, not inline (TU)  : 15.5201 ms
member function (inline) : 12.3991 ms
public getters (inline)  : 12.4748 ms
```

Version	-O0 (ns per call)	-O2 (ns per call)
Friend, inline	6.0	1.25
Friend, not inline (separate source file)	5.8	1.55
Member function	6.0	1.24
Public getters	11.9	1.25

Explanation of the result:

- **At -O0 the inline keyword has no visible effect** (60.2 ms against 58.0 ms; the small difference is measurement noise). With optimisation off, GCC does not substitute function bodies, so every version is really called. The getters version is about twice as slow because each of its four getter calls is an actual function call that is not removed.
- **At -O2 the inlined versions take about 20% less time** than the version in a separate source file (12.5 ms against 15.5 ms), because the call is removed and the optimiser can combine the body with the surrounding loop. The getters and the member function become as fast as the friend, because the compiler inlines the tiny getters, so the extra function calls disappear completely.
- **Absolute cost:** even the slowest -O2 version spends about 1.5 ns per call, so for millions of calls inline gains milliseconds, not seconds. The gain is real but small; the correct reason to choose a design is encapsulation first and speed second.

An earlier version of the benchmark built the two Time objects with remainder operations inside the timed loop. The arithmetic dominated and hid the call cost (32.9 ms inline against 30.9 ms not inline at -O2), so the loop was changed to use a pre-built array. This shows that a measurement must isolate the thing being measured.

6.5 Deployment decision

Decision: keep minutesBetween() and earlier() as friend functions, defined inline in the header, and do not add public getters for h and m.

- They work symmetrically on two objects and are called in the natural form minutesBetween(a, b).

- They do not enlarge the public interface, so the internal representation (hours and minutes) can still be changed, and no outside code can obtain or alter h and m.
- The measurements show that performance does not decide the matter at -O2: friend, member and getter versions run in 12.4 to 12.5 ms, so the choice can be made on design grounds. Only the friend that is hidden in a separate source file was slower (15.5 ms), so the definitions should be in the header and marked inline.

If more functions need the data, the friend list would grow; in that case a small member such as minutesSinceMidnight() const would be a better single access point, because it exposes a meaningful value and not the representation.

6.6 Alternative Concept and Complexity Analysis

Design	**Call syntax**	**Time per call**	**Measured at -O0 / -O2**	**Encapsulation**
Friend functions	minutesBetween(a, b)	O(1)	6.0 ns / 1.25 ns	Private data stays private; the friend list is part of the class design
Member functions	a.minutesTo(b)	O(1)	6.0 ns / 1.24 ns	Equally good; asymmetrical in form
Public getters with non-member functions	minutesBetweenGetter(a, b)	O(1)	11.9 ns / 1.25 ns	Weakest: h and m become part of the public interface

All three designs take O(1) time and O(1) space per call: a few additions, one multiplication and one comparison. After inlining at -O2 the three are indistinguishable (1.24 to 1.25 ns per call), and at -O0 the getters version is about twice as slow. The designs therefore differ in encapsulation and style and not in complexity.

Evidence summary: a justified friend-function design, correctness tests, measured inline timings at two optimisation levels and an evidence-based decision.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected program: Q4 – Savings Account Mini-Statement (fixed array with shifting)

The program chosen from Q1 to Q6 is the SavingsAccount of Q4, because its design shows a limitation that appears when the requirement changes (10 transactions become 50 or more) and the data grows (many transactions are recorded). The redesign keeps the same class interface and changes only how the history is stored.

A. Diagnose the Limitation (2 marks)

In the original design record() keeps the array ordered oldest-first. When the array is full, every new transaction moves the other MAXTX − 1 entries one place to the left before the new one is stored. The cost of one transaction is therefore O(k) with k = MAXTX − 1, and it grows with the capacity, although the only information that changes is one entry.

Evidence from the original program (Q7/q7_bench.cpp, class ShiftAccount, -O2, 1,000,000 transactions for capacities 10, 50 and 500 and 200,000 for 5000):

Capacity N	**Time per transaction, original (ns)**	**Ratio to N = 10**
10	9.3	1.0
50	20.4 (17.9 in repeat run)	2.2
500	61.5 (82.2 in repeat run)	6.6
5000	1511 (1577 in repeat run)	163

The cost rises with N (about 1.5 microseconds per transaction at N = 5000), which is the linear growth that the analysis in Q4 predicts. The cause is a characteristic of the design: the logical order of the transactions (oldest to newest) is tied to their physical positions in the array, so discarding the oldest one forces every other element to move. The Q4 requirement to keep 50 transactions already costs about twice as much per transaction as 10.

B. Define the Design Objective (2 marks)

Objective: the cost of recording a transaction must not depend on the capacity N. Measured at -O2, the redesigned version must satisfy all of the following:

- time per transaction at N = 5000 is within 2 times the time per transaction at N = 10 (the original is 163 times);
- at least 10 times faster than the original at N = 500 and at least 100 times faster at N = 5000;
- the mini-statement is identical to that of the original for the same sequence of transactions, the public interface does not change, and no additional array or heap memory is used.

C. Design the Revised Solution (4 marks)

What changes: the history is stored in a circular buffer inside the same private array. A new private member start holds the index of the oldest entry. A new transaction is written at position (start + count) % MAXTX. When the array is full, the oldest entry is overwritten and start moves on by one (modulo MAXTX). No element is moved. miniStatement() prints from start and wraps around, so the output order (oldest first) is unchanged.

C++ feature used: private data with member functions, so that the change is hidden from callers (encapsulation, as argued in Q4), the modulo operator for the wrap-around and a class template parameter in the benchmark so that the same code is tested with several capacities.

Extra memory: one int (start), 4 bytes per account. The array stays at MAXTX × 16 bytes.

New trade-offs: the reading code is a little harder (the index wraps); the modulo operation is slightly more costly than a plain increment for a single step, which matters only when N is very small; and any code that read tx[0] as the oldest entry must now use (start + i) % MAXTX.

Class diagram

SavingsAccount (redesigned)
– accNo : string – balance : double – tx[MAXTX] : Transaction – start : int (new: index of the oldest entry) – count : int

SavingsAccount (redesigned)
+ SavingsAccount(no, opening)
+ deposit(amount) : void
+ withdraw(amount) : bool
+ miniStatement() const : void
– record(type, amount) : void (O(1) instead of O(k))

Pseudocode

```
record(type, amount):
    pos = (start + count) mod MAXTX
    tx[pos] = (type, amount)
    if count < MAXTX:
        count = count + 1
    else:
        start = (start + 1) mod MAXTX        // the oldest entry has just been overwritten


miniStatement():
    for i = 0 to count - 1:
        print tx[(start + i) mod MAXTX]
```

D. Predict Before Implementing (1 mark)

Prediction: the redesigned version will take roughly the same time per transaction for every capacity (a few nanoseconds, from the store, the index update and one modulo), while the original grows in proportion to N. I therefore expect the speed-up to grow with N: about 2 times at N = 10 (only 9 elements are shifted), at least 10 times at N = 500 and at least 100 times at N = 5000. The measure is nanoseconds per transaction at -O2, and the statements of the two versions must be identical.

E. Implement and Validate the Redesign (2 marks)

The original (ShiftAccount) and the redesign (RingAccount) were run on the same sequence of operations (every third operation a withdrawal, the others deposits) in the same program, and their mini-statements were compared element by element. The source is Q7/q7_bench.cpp (in the repository).

Measured results at -O2 (g++ -std=c++17 -O2)

```
N=10  T=1000000  shift: 9.26426 ms (9.26426 ns/tx)  ring: 4.48854 ms (4.48854 ns/tx)  speed-up 2.06398x
statements identical: true
N=50  T=1000000  shift: 20.3907 ms (20.3907 ns/tx)  ring: 5.3366 ms (5.3366 ns/tx)  speed-up 3.82092x
statements identical: true
N=500  T=1000000  shift: 61.5039 ms (61.5039 ns/tx)  ring: 4.38908 ms (4.38908 ns/tx)  speed-up 14.0129x
statements identical: true
N=5000  T=200000  shift: 302.213 ms (1511.06 ns/tx)  ring: 0.786017 ms (3.93009 ns/tx)  speed-up 384.486x
statements identical: true
```

Capacity N	**Original (ns/tx)**	**Redesigned (ns/tx)**	**Speed-up, -O2 (first run / repeat)**	**Speed-up, -O0**
10	9.3	4.5	2.1x / 2.2x	3.7x
50	20.4	5.3	3.8x / 4.4x	9.7x
500	61.5	4.4	14.0x / 16.0x	72.6x
5000	1511	3.9	384x / 395x	827x

In every row the statements of the two versions were identical (statements identical: true), so the redesign behaves the same as the original.

F. Defend the Final Design (2 marks)

- **Was the objective achieved?** Yes. At -O2 the redesigned version took 3.9 ns per transaction at N = 5000 against 4.5 ns at N = 10, well inside the target of 2 times (the original grew 163 times). The speed-up at N = 500 was 14 to 16 times (target at least 10) and at N = 5000 it was 384 to 395 times (target at least 100). The interface and the output are unchanged and no additional array was needed. The prediction of roughly 2 times at N = 10 was also confirmed (2.1 to 2.2 times).
- **New costs:** one extra int per account, a modulo in each record and each printed line, and code that is harder to read and to debug because the position of the oldest entry is no longer fixed at index 0.
- **Where the original may still be preferable:** for the actual ATM requirement of 10 transactions the original costs only about 9 ns per transaction, which is irrelevant at the

speed at which people use an ATM, and its code is the simplest to read and check. The saving matters for the 50-transaction requirement only if many transactions are processed in bulk, for example when a day's transactions for millions of accounts are replayed, and it matters most for large N. The two versions have the same O(N) space.

- **Limits of this evidence:** the timings were taken on a shared machine and vary by up to about 30% between runs (for example 61.5 ms and 82.2 ms for the original at N = 500), but the ratios and the linear growth were the same in every run. The benchmark uses simple transaction values and only deposits and withdrawals.

Conclusion: the circular buffer is the appropriate redesign when the capacity is larger than about 50 or when many transactions are processed; the simple shifting array is acceptable and easier to maintain at 10.